

Working with Data

Session 2 — What happens when tables meet?

Block 3 — Joins and grain: keeping the row count honest

Knowledge check 2 — 10 minutes, closed book, 3 questions

- Covers Session 1 and Homework 1: `COUNT` , `NULL` in a `WHERE` , `GROUP BY` , `ORDER BY` and `LIMIT`
- Laptops closed. Paper only.
- One scenario, three questions. Each question stands on its own.
- Worth 5% of the final grade
- **This block's lecture is 35 minutes**, not 45 — the check comes out of lecture, never out of lab
- Hand it in when the timer goes. Then we begin.

This block in one sentence

A join is a claim about how two grains relate.

Count the rows before. Count the rows after. Explain every row that appeared or disappeared.

What we cover

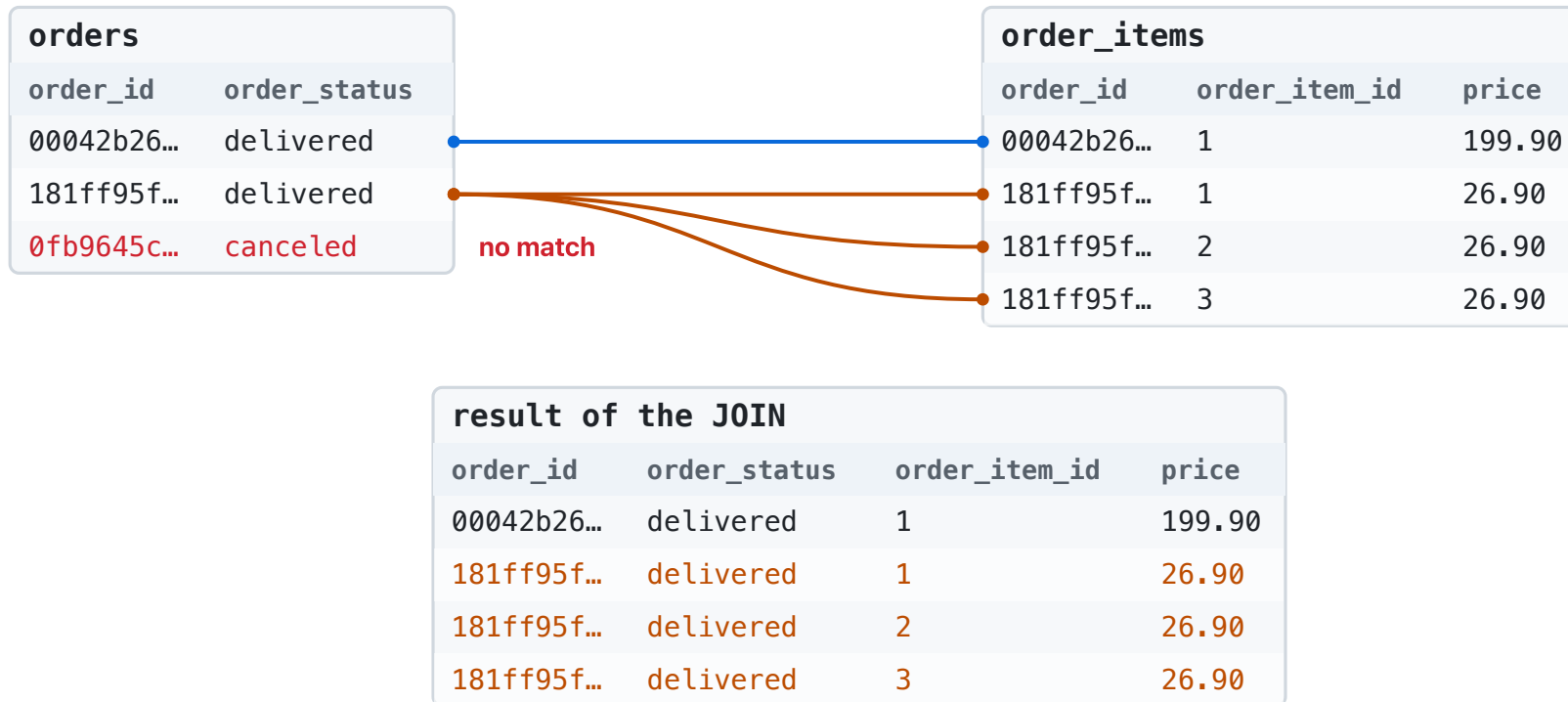
1. Why the data lives in several tables
2. The three counts after every join
3. `INNER` against `LEFT`, and the anti-join
4. `HAVING`: the keys that repeat — and fan-out, the silent killer
5. Two patterns: no join, or aggregate first and then join — with a `CTE`
6. Git thread and AI thread
7. Stretch — start the Lab 3 clone — then Lab 3

Why the data lives in several tables

- An order is stored **once**, in `orders`. Its items live in `order_items`. Its customer lives in `customers`.
- A **primary key** says which row is which: `order_id` in `orders`; `(order_id, order_item_id)` in `order_items`.
- A **foreign key** points at another table's key: `order_items.order_id` points at `orders.order_id`.

Relationship	In this file	Each left row matches
one-to-one	an order and its <code>customers</code> row	exactly one row
one-to-many	an order and its items	zero, one, or several rows
many-to-many	reviews and orders (Homework 2)	several, from both sides

The picture for the whole block



rows in orders: 3 · rows in the result: 4 · distinct order_id in the result: 2

Three real orders from the Lab 3 files, all from São Paulo. Key lines from `orders` to `order_items`. The row count goes under the result.

Predict, then watch: the three counts

Cell 1 of the lecture notebook:

```
SELECT (SELECT COUNT(*) FROM orders) AS rows_in_orders,  
       COUNT(*)                      AS rows_in_result,  
       COUNT(DISTINCT o.order_id)    AS distinct_orders_in_result  
FROM orders AS o  
JOIN order_items AS i ON i.order_id = o.order_id
```

Predict: `orders` has 2,653 rows. Will the result have more rows than that, or fewer? How many distinct orders will be in it?

The three counts — what it printed

cell 1

```
con.sql("""
    SELECT (SELECT COUNT(*) FROM orders) AS rows_in_orders,
           COUNT(*)                      AS rows_in_result,
           COUNT(DISTINCT o.order_id)    AS distinct_orders_in_result
    FROM orders AS o
    JOIN order_items AS i ON i.order_id = o.order_id
""").df()
```

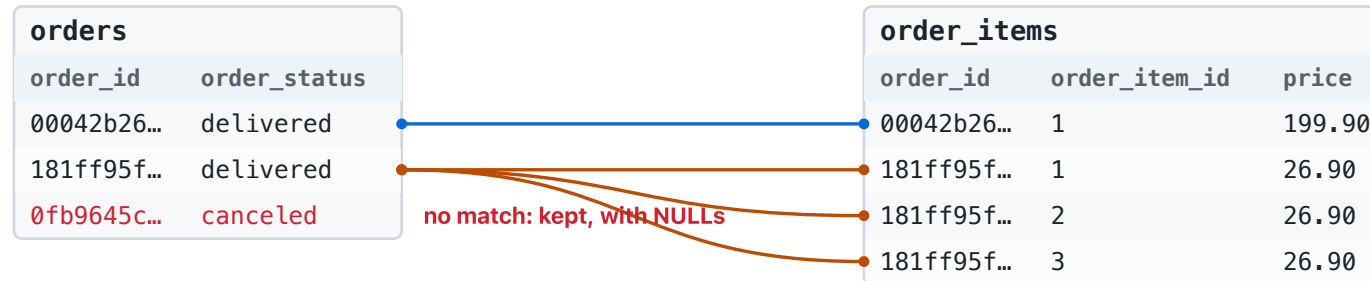
	rows_in_orders	rows_in_result	distinct_orders_in_result
0	2653	3024	2610

Reading the three counts

Count	<code>orders</code> → <code>order_items</code>	What it tells you
rows in the left table	2,653	what went in
rows in the result	3,024	more than distinct orders: some orders matched several items
distinct left keys in the result	2,610	fewer than went in: 43 orders matched nothing , and the join dropped them

- `result = left = distinct`: every left row matched exactly once. Nothing multiplied, nothing lost.
- `result > distinct`: a left row matched several rows. **This is where multiplication comes from.**
- `distinct < left`: some left rows matched nothing. `JOIN` (that is, `INNER JOIN`) drops them without a word.

INNER drops the unmatched; LEFT keeps them



result of the LEFT JOIN			
order_id	order_status	order_item_id	price
00042b26...	delivered	1	199.90
181ff95f...	delivered	1	26.90
181ff95f...	delivered	2	26.90
181ff95f...	delivered	3	26.90
0fb9645c...	canceled	NULL	NULL

rows in orders: 3 · rows in the result: 5 · distinct order_id in the result: 3

Predict, then watch (cell 2): the same three counts with LEFT JOIN . Predict the last two numbers.

INNER drops the unmatched; **LEFT** keeps them — what it printed

cell 2

```
con.sql("""
    SELECT (SELECT COUNT(*) FROM orders) AS rows_in_orders,
           COUNT(*)                      AS rows_in_result,
           COUNT(DISTINCT o.order_id)    AS distinct_orders_in_result
    FROM orders AS o
    LEFT JOIN order_items AS i ON i.order_id = o.order_id
""").df()
```

	rows_in_orders	rows_in_result	distinct_orders_in_result
0	2653	3067	2653

Predict, then watch: the anti-join

Cell 3:

```
SELECT COUNT(*) AS orders_with_no_items
FROM orders AS o
LEFT JOIN order_items AS i ON i.order_id = o.order_id
WHERE i.order_id IS NULL
```

Predict the count. Then watch cell 4: what are those orders? (their `order_status`)

The anti-join — what it printed

cell 3

```
con.sql("""  
    SELECT COUNT(*) AS orders_with_no_items  
    FROM orders AS o  
    LEFT JOIN order_items AS i ON i.order_id = o.order_id  
    WHERE i.order_id IS NULL  
""").df()
```

	orders_with_no_items
0	43

The orders with no items, by status

```
cell 4 con.sql("""
SELECT o.order_status, COUNT(*) AS orders
FROM orders AS o
LEFT JOIN order_items AS i ON i.order_id = o.order_id
WHERE i.order_id IS NULL
GROUP BY o.order_status
ORDER BY orders DESC
""").df()
```

	order_status	orders
0	canceled	25
1	unavailable	17
2	invoiced	1

- **RIGHT JOIN** keeps every row of the right table instead; **FULL JOIN** keeps every row of both. Write a **LEFT JOIN** from the side you mean.
- **USING (order_id)** is short for **ON i.order_id = o.order_id** when the names match. Join on the wrong column and you get zero matches, or every row paired with every row. Neither is an error.

One order's items

Predict, then watch (cell 5): the items of order `181ff95f...`, the orange one in the picture, with three lines.

```
SELECT order_id, order_item_id, product_id, price
FROM order_items
WHERE order_id LIKE '181ff95f%'
ORDER BY order_item_id
```

Predict: three rows. Three different products, or the same product three times?

One order's items — what it printed

```
cell 5 con.sql("""  
    SELECT order_id, order_item_id, product_id, price  
    FROM order_items  
    WHERE order_id LIKE '181ff95f%'  
    ORDER BY order_item_id  
    """).df()
```

	order_id	order_item_id	product_id	price
0	181ff95f97fee531c4afa9e882186f18	1	585e7f7e97e55cb9e74556236b43b0be	26.9
1	181ff95f97fee531c4afa9e882186f18	2	585e7f7e97e55cb9e74556236b43b0be	26.9
2	181ff95f97fee531c4afa9e882186f18	3	585e7f7e97e55cb9e74556236b43b0be	26.9

Which orders have more than one item?

Predict, then watch (cell 5b). One row per order, and keep the orders with more than one item:

```
SELECT order_id, COUNT(*) AS items
FROM order_items
WHERE COUNT(*) > 1
GROUP BY order_id
```

Predict: does it run?

Which orders have more than one item? — what it printed

cell 5b

```
try:
    con.sql("""
        SELECT order_id, COUNT(*) AS items
        FROM order_items
        WHERE COUNT(*) > 1
        GROUP BY order_id
    """).df()
except duckdb.Error as e:
    print("DuckDB refused:", str(e).splitlines()[0])
```

DuckDB refused: Binder Error: WHERE clause cannot contain aggregates!

The same condition in **HAVING**

cell 5c

```
con.sql("""
    SELECT order_id, COUNT(*) AS items
    FROM order_items
    GROUP BY order_id
    HAVING COUNT(*) > 1
    ORDER BY items DESC, order_id
    LIMIT 5
    """).df()
```

	order_id	items
0	ab14fdcfbe524636d65ee38360e22ce8	20
1	be382a9e1ed25128148b97d6bfdb21af	8
2	62073ec6b54b8e6322037fc0f3591ad3	7
3	28cf49bbfe2683759d3cf866b7fda22f	6
4	3e7bc2221b2c46fafa06f741386784fc	6

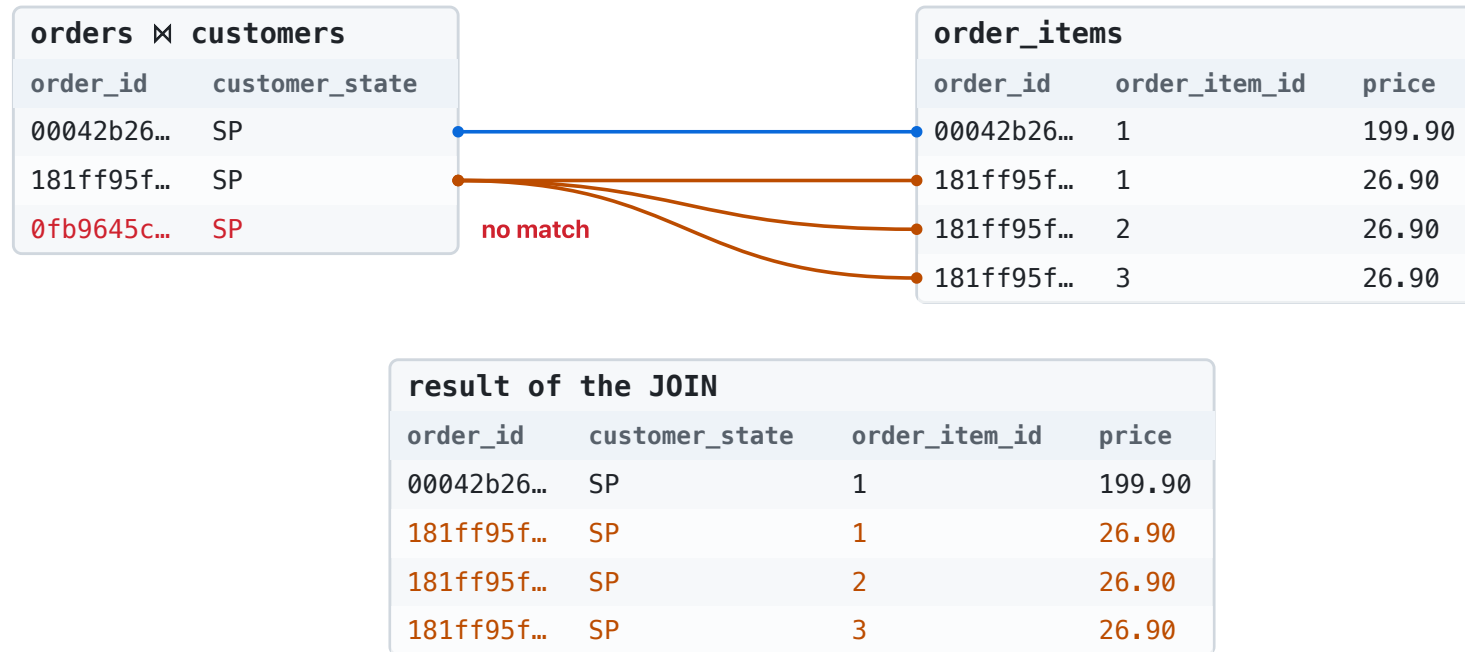
How many orders repeat

cell 5d

```
con.sql("""  
    SELECT COUNT(*) AS orders_that_repeat, SUM(items) AS their_item_rows  
    FROM (SELECT order_id, COUNT(*) AS items  
          FROM order_items  
          GROUP BY order_id  
          HAVING COUNT(*) > 1)  
""").df()
```

	orders_that_repeat	their_item_rows
0	265	679.0

Fan-out, the silent killer



orders in SP: 3 · rows for SP after the join: 4 · COUNT(*) now counts items

Orders per customer state, no items

Watch (cell 6): orders joined to customers, no items. São Paulo, **SP**, comes first.

```
cell 6 con.sql("""  
    SELECT c.customer_state, COUNT(*) AS orders  
    FROM orders AS o  
    JOIN customers AS c ON c.customer_id = o.customer_id  
    GROUP BY c.customer_state  
    ORDER BY orders DESC  
    LIMIT 5  
""").df()
```

	customer_state	orders
0	SP	1133
1	RJ	348
2	MG	302
3	RS	139
4	PR	122

Predict, then watch: the same count, after one more join

Cell 7: cell 6's query with one more line.

```
SELECT c.customer_state, COUNT(*) AS orders
FROM orders AS o
JOIN customers AS c ON c.customer_id = o.customer_id
JOIN order_items AS i ON i.order_id = o.order_id
GROUP BY c.customer_state
ORDER BY orders DESC
LIMIT 5
```

Predict: for SP, bigger than 1,133, smaller, or the same?

The same count, after one more join — what it printed

cell 7

```
con.sql("""
    SELECT c.customer_state, COUNT(*) AS orders
    FROM orders AS o
    JOIN customers AS c ON c.customer_id = o.customer_id
    JOIN order_items AS i ON i.order_id = o.order_id
    GROUP BY c.customer_state
    ORDER BY orders DESC
    LIMIT 5
    """).df()
```

	customer_state	orders
0	SP	1278
1	RJ	432
2	MG	338
3	RS	149
4	PR	139

customers : one row is one what?

Predict, then watch (cell 8):

```
SELECT COUNT(*), COUNT(DISTINCT customer_id), COUNT(DISTINCT customer_unique_id)
FROM customers
```

Predict: one row, three numbers. Are they equal? If not, which is smallest?

customers — what it printed

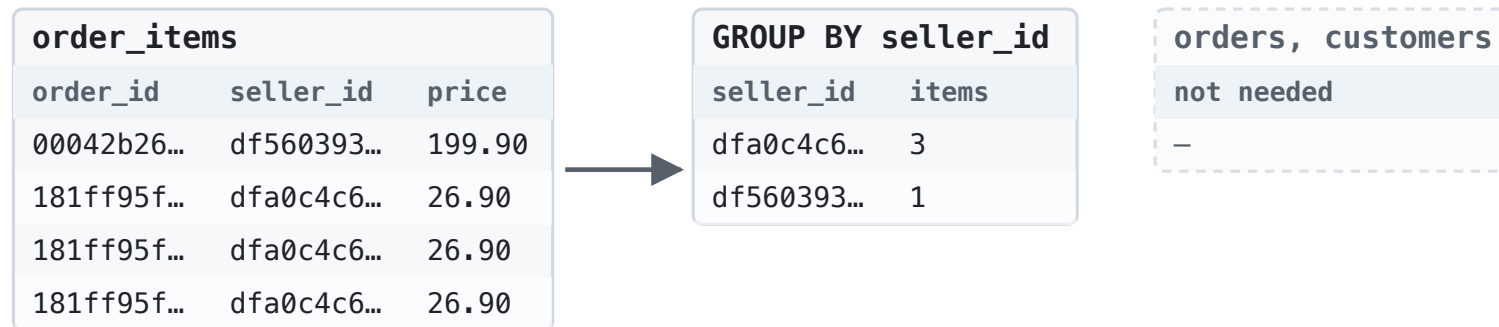
cell 8

```
con.sql("""
    SELECT COUNT(*)                AS n_rows,
           COUNT(DISTINCT customer_id) AS n_customer_id,
           COUNT(DISTINCT customer_unique_id) AS n_customer_unique_id
    FROM customers
""").df()
```

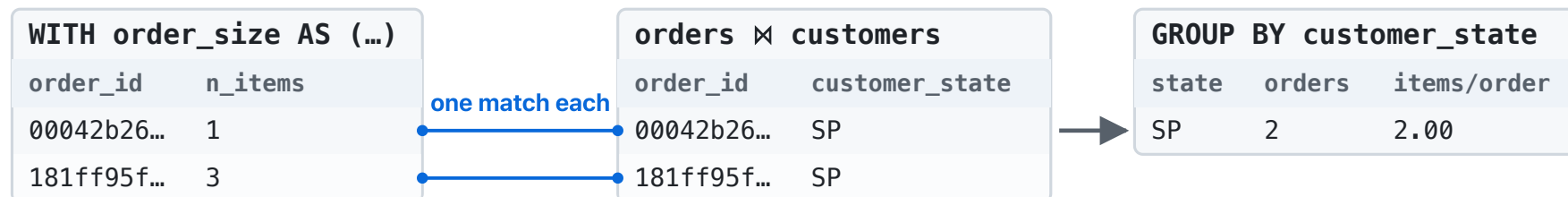
	n_rows	n_customer_id	n_customer_unique_id
0	2653	2653	2634

Aggregate at the grain of the measure; join only what does not multiply it

(a) the dimension is on the measure's own table → no join



(b) the dimension is on a parent → aggregate to its grain first, then join



rows going in: 2 · rows coming out: 2 · distinct orders: 2

Unnecessary joins add dependencies. Multiplication happens when a left row has several matches.

Pattern (a): items per seller, no join

Watch (cell 9): `order_items` alone, the five busiest sellers.

Unnecessary joins add dependencies. Multiplication happens when a left row has several matches.

cell 9

```
con.sql("""
    SELECT seller_id, COUNT(*) AS items
    FROM order_items
    GROUP BY seller_id
    ORDER BY items DESC, seller_id
    LIMIT 5
""").df()
```

	seller_id	items
0	1025f0e2d44d7041d6cf58b6550e0bfa	57
1	7c67e1448b00f6e969d365cea6b010ab	53
2	cc419e0650a3c5ba77189a1882b7556a	53
3	da8622b14eb17ae2831f4ac5b9dab84a	53
4	1f50f920176fa81dab994f9023523100	44

Predict, then watch: pattern (b), with a CTE

```
WITH order_size AS (  
    SELECT order_id, COUNT(*) AS n_items  
    FROM order_items  
    GROUP BY order_id  
)  
SELECT c.customer_state, COUNT(*) AS orders, ROUND(AVG(s.n_items), 2) AS items_per_order  
FROM order_size AS s  
JOIN orders AS o ON o.order_id = s.order_id  
JOIN customers AS c ON c.customer_id = o.customer_id  
GROUP BY c.customer_state  
ORDER BY orders DESC  
LIMIT 5
```

`WITH name AS (...)` is a **CTE**: a query with a name, used by the query below it. Cell 10; then cell 11, its three counts. **Predict**: how many rows does `order_size` have? How many orders will SP show?

Pattern (b), with a CTE — what it printed

cell 10

```
con.sql("""
    WITH order_size AS (
        SELECT order_id, COUNT(*) AS n_items
        FROM order_items
        GROUP BY order_id
    )
    SELECT c.customer_state, COUNT(*) AS orders, ROUND(AVG(s.n_items), 2) AS items_per_order
    FROM order_size AS s
    JOIN orders AS o ON o.order_id = s.order_id
    JOIN customers AS c ON c.customer_id = o.customer_id
    GROUP BY c.customer_state
    ORDER BY orders DESC
    LIMIT 5
""").df()
```

	customer_state	orders	items_per_order
0	SP	1116	1.15
1	RJ	342	1.26
2	MG	295	1.15
3	RS	137	1.09
4	PR	121	1.15

Pattern (b): the three counts for that join

```
cell 11 con.sql("""  
    WITH order_size AS (SELECT order_id, COUNT(*) AS n_items FROM order_items GROUP BY order_id)  
    SELECT (SELECT COUNT(*) FROM order_size) AS rows_going_in,  
           COUNT(*)                          AS rows_coming_out,  
           COUNT(DISTINCT s.order_id)        AS distinct_orders_out  
    FROM order_size AS s  
    JOIN orders AS o    ON o.order_id = s.order_id  
    JOIN customers AS c ON c.customer_id = o.customer_id  
    """).df()
```

	rows_going_in	rows_coming_out	distinct_orders_out
0	2610	2610	2610

"Aggregate, then join" is pattern (b), not a cure

- Used on a pattern (a) question, it throws the dimension away: items per seller needs `seller_id`, and one row per order has none.
- Join an order's total back to that order's items and the total repeats once per item: the same fan-out, from the other side.
- `COUNT(DISTINCT ...)` can make a count come out right after a fan-out. It hides why it was wrong. `SUM(DISTINCT ...)` adds up distinct *values*, not distinct rows.
- The question decides the pattern: which table holds the measure, and which holds the dimension?

Git thread: the three counts go in the commit message

```
git add notebooks/report.ipynb
git commit -m "Items per order by state: count items per order first" \
    -m "order_size 2,610 rows -> joined 2,610 rows, 2,610 distinct orders"
```

- Any commit that **changes a join** carries its three counts. A second `-m` becomes a second paragraph; the `\` at the end of the line continues the command on the next.
- In six months, `git log` tells you what the join did on the day you wrote it.

Watch now; you commit in the lab, with your own join's counts.

AI thread: AI writes SQL that runs. Running is not right.

From minute 20 of the lab, if you use an assistant:

1. Give it the schema (paste `DESCRIBE`) and the grain of each table, in words.
2. Give it the number you expected and the number you got.
3. Ask for the checks that would tell them apart, not for the fix.
4. Run the checks yourself. Paste what they returned.
5. Make one change. Reconcile. Explain it in your own words.

The assistant's join returns rows. Whether the rows are tripled is a count only you can run.

5-minute stand-and-stretch

Start the Lab 3 clone and `uv sync`, then stand up — it downloads while you stretch.

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5294-lab03-joins.git
cd ecbs5294-lab03-joins
uv sync
```

Then leave the laptop. Back in five.

Lab 3 — The revenue that grew when we joined

First 20 minutes: no AI. Chat windows and assistant panels closed; VS Code stays open. Search engines are fine.

Why: counting the rows before and after a join has to become a reflex, and a reflex comes from doing it, not from asking for it. The 20 minutes cover the diagnosis and the first query you write. The exam has no chat window.

TAs will ask you to close a window. That is not a penalty.

The symptom, and how to start

The first minute: check the clone ran (the `ecbs5294-lab03-joins` folder, `uv sync` finished). Open that folder in VS Code, open `notebooks/report.ipynb`, pick the `.venv` kernel, run all. Then read the README.

- The report's total revenue: **364,110.54 BRL**
- Its revenue by payment type adds up to **383,270.23** — more than the total
- Its revenue by category adds up to **351,299.08** — less than the total

The core is the table that comes out **above** the total: diagnose it (section A), write sections B and C yourself, run the check (D). **Start the note by minute 22:** it gets five minutes. Section E is for after the note; the category table is the stretch. At home: the README's *Get the project* has the `git clone` line.

The last ten minutes — explain it to a neighbour

Speaker, about a minute, pointing at the screen, not reading the note: symptom → cause → evidence → change → verification.

Listener asks:

1. Show me the query that proves it.
2. Why was it wrong, not just where?
3. One order in `orders` has no payment row. If your state table starts from the payments, is that order in it? Which query would show it?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` to the Lab 3 checkpoint on Moodle.